

TRABAJO PRÁCTICO INTEGRADOR

Estructuras de Datos, Contrato Equals/HashCode y Complejidad Algorítmica

Asignatura: Desarrollo de Software

INTEGRANTES

Beltran Nadia

Canales Juan Ignacio

Celi Leonardo

López Noelia

Ochoa Melanie

Santibañez Guada

Criterio de Evaluación y Política de IA: Se permite el uso de herramientas de Inteligencia Artificial (LLMs). Sin embargo, el práctico está diseñado para evaluar el **razonamiento crítico, el análisis de estados de memoria y la justificación técnica**. Las respuestas genéricas generadas por IA sin prueba de trazabilidad manual o sin responder a los escenarios de borde específicos serán penalizadas.

Criterios Generales de Entrega

- Entregar un informe en formato PDF publicado en un repositorio de GitHub.
- Incluir fragmentos de código fuente en Java y los diagramas/trazas explicativas solicitadas.

Consigna 1: Diagnóstico de Bug Silencioso y Análisis de Memoria

Un equipo de desarrollo junior implementó el siguiente sistema para gestionar el acceso de empleados a un edificio mediante tarjetas de identificación mutable. El código no lanza excepciones en tiempo de ejecución, pero presenta comportamientos anómalos en producción.

```
import java.util.*;

class TarjetaAcceso {
    private String codigo;
    private int nivelAcceso;

    public TarjetaAcceso(String codigo, int nivelAcceso) {
        this.codigo = codigo;
        this.nivelAcceso = nivelAcceso;
    }

    public void setNivelAcceso(int nivelAcceso) {
        this.nivelAcceso = nivelAcceso;
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (o == null || getClass() != o.getClass()) return false;
        TarjetaAcceso tarjeta = (TarjetaAcceso) o;
        return nivelAcceso == tarjeta.nivelAcceso && Objects.equals(codigo,
tarjeta.codigo);
    }

    @Override
    public int hashCode() {
        return Objects.hash(codigo, nivelAcceso);
    }
}

public class ControlAcceso {
```

```

public static void main(String[] args) {
    Set<TarjetaAcceso> tarjetasValidas = new HashSet<>();[cite: 3]
    TarjetaAcceso t1 = new TarjetaAcceso("ACC-101", 2);

    tarjetasValidas.add(t1);

    // Modificación posterior a la inserción
    t1.setNivelAcceso(5);

    System.out.println("¿Tarjeta válida en sistema?: " +
tarjetasValidas.contains(t1));
    System.out.println("Cantidad de tarjetas registradas: " +
tarjetasValidas.size());
}
}

```

Responder y Justificar:

1. **Traza del Estado Interno:** Explique paso a paso qué ocurre dentro de la estructura HashSet:

- Al ejecutar tarjetasValidas.add(t1).

Al añadir el objeto t1 al hashSet, Java calcula su t1.hashCode() correspondiéndole un número entero. Luego, la estructura utiliza el entero devuelto por hashCode() para calcular en qué índice del cajón (bucket) exacto se almacenará el elemento.

- Al ejecutar t1.setNivelAcceso(5).

Se modifica el estado del objeto, por lo que el valor devuelto por futuras llamadas a t1.hashCode() cambia de valor. El objeto físicamente sigue guardado en el cajón (bucket) del hash original, pero la colección no se enteró de esta modificación.

- Al ejecutar tarjetasValidas.contains(t1).

Java vuelve a pedir el hashCode() al objeto para saber en qué cajón buscar. Como se cambió el número entero por ende cambiando el cajón, Java va a buscar en un cajón vacío devolviendo false.

2. **Explicación del Cajón (Bucket):** ¿En qué número/ubicación de cajón quedó guardado el objeto y en cuál lo busca Java al hacer contains()?

Al insertar t1, el objeto queda almacenado en el cajón (bucket) que se calcula a partir de hashCode(), con sus atributos originales (ACC-101 , 2). Físicamente el objeto sigue en el mismo lugar, pero al cambiar con setNivelAcceso(5) su nivel de 2 a 5, tarjetasValidas.contains(t1) busca el objeto usando los atributos (ACC-101 , 5) lo que lo envía a un cajón vacío, perdiendo así el acceso al objeto original.

3. **Fuga de Memoria (Memory Leak):** ¿Por qué este patrón genera una fuga de memoria si se ejecuta repetidamente en un servidor que opera 24/7?

El patrón genera una fuga de memoria porque los objetos que fueron modificados no se pueden borrar con .remove() ni son eliminados por el recolector de basura debido a que el Hashset conserva una referencia viva al objeto.

4. **Refactorización Obligatoria:** Aplique dos soluciones arquitectónicas distintas para corregir el problema (una basada en inmutabilidad y otra ajustando el criterio de la entidad).

- Inmutabilidad: Se podría borrar los métodos setter y además declarar como final los atributos utilizados por equals() y hashCode() para no cambiar el valor del código hash.
- Ajustando el criterio de la entidad: Se podría agregar un identificador inmutable (ej. private final Long id) para calcular en hashCode() y equals() únicamente sobre este nuevo atributo y no sobre los atributos cambiantes (nivel de acceso).

(salida del código)

```

¿Tarjeta válida en sistema?: false
Cantidad de tarjetas registradas: 1

```

Consigna 2: Selección y Diseño de Arquitectura de Colecciones

Diseñe el backend de un módulo para una plataforma de E-Commerce. Debe elegir la interfaz y la implementación concreta del Java Collections Framework (java.util) más eficiente para cada una de las siguientes tres funcionalidades:

Requerimiento 1: Sistema de Auditoría de Transacciones
Registrar operaciones en el orden exacto en que ocurren. Se requiere insertar miles de registros por segundo al final de la colección y recorrerlos secuencialmente.

Requerimiento 2: Catálogo de Ofertas Relámpago
Mantener los productos ordenados automáticamente por precio de menor a mayor. Debe permitir consultar eficientemente el producto más barato y el más caro.

Requerimiento 3: Búsqueda de Sesiones de Usuarios Activos
Buscar instantáneamente la sesión de un usuario dado su Token (String único) entre 500,000 usuarios conectados en simultáneo.

Responder y Justificar:

Completar la siguiente tabla comparativa y justificar la decisión con la complejidad algorítmica Big O:

Requerimiento	Interfaz Seleccionada (List , Set , Map , Queue) PDF	Implementación Concreta (ArrayList , TreeSet , HashMap , etc.) PDF	Complejidad Búsqueda	Complejidad Inserción	Justificación Técnica de la Elección
1. Auditoría					
2. Ofertas					
3. Sesiones					

Requerimiento	Interfaz seleccionada	Implementación concreta	Complejidad Búsqueda	Complejidad Inserción	Justificación Técnica de la Elección
1. Auditoría	List	ArrayList	$O(1)$ acceso por índice / $O(n)$ búsqueda por valor (no crítica)	$O(1)$ promedio	Para este sistema necesitamos inserciones rápidas al final porque se registran miles de transacciones por segundo. ArrayList tiene inserción al final $O(1)$ promedio y mejor localidad de caché que LinkedList para recorridos secuenciales. Aunque LinkedList también inserta en $O(1)$ al final, el recorrido es más lento en la práctica.
2. Ofertas	Set (SortedSet)	TreeSet con comparator por precio	$O(1)$ para <code>first()</code> / <code>last()</code> (más barato/más caro) / $O(\log n)$ para búsqueda por precio	$O(\log n)$	Necesitamos mantener los productos siempre ordenados por precio. TreeSet (implementado con árbol rojo-negro) mantiene el orden automáticamente. Permite obtener el más barato (<code>first()</code>) y el más caro (<code>last()</code>) en $O(1)$, e inserciones en $O(\log n)$. PriorityQueue no sirve porque solo da acceso al más barato.
3. Sesiones	Map	HashMap <String, Sesion>	$O(1)$ promedio	$O(1)$ promedio	Para la búsqueda de sesiones entre 500,000 usuarios, necesitamos acceso instantáneo por Token. HashMap ofrece $O(1)$ promedio en inserción y búsqueda, mucho más rápido que TreeMap ($O(\log n)$) o una búsqueda lineal en ArrayList ($O(n)$). El Token es único, ideal como clave.

Consigna 3: Optimización Algorítmica ($O(n^2)$ -> $O(n)$)

Un desarrollador escribió el siguiente método para encontrar el primer carácter que no se repite dentro de una cadena de texto (ejemplo: en "barco", el primero es 'b'; en "abarca", el primero es 'r').

```
public static Character primerCaracterNoRepetidoLento(String texto) {  
    // Algoritmo ineficiente  $O(n^2)$   
    for (int i = 0; i < texto.length(); i++) {  
        char c = texto.charAt(i);  
        boolean duplicado = false;  
        for (int j = 0; j < texto.length(); j++) {  
            if (i != j && c == texto.charAt(j)) {  
                duplicado = true;  
                break;  
            }  
        }  
        if (!duplicado) return c;  
    }  
    return null;  
}
```

Responder y Justificar:

1. **Refactorización a $O(n)$:** Escriba una versión optimizada del método utilizando la colección adecuada de java.util (HashMap, LinkedHashMap, o HashSet) que resuelva el problema realizando como máximo dos recorridos lineales ($O(n)$).

```
public class PrimerCaracterNoRepetido {  
    public static Character primerCaracterNoRepetido(String texto) {  
        Map<Character, Integer> frecuencias = new HashMap<>();  
  
        // Primer recorrido: contar frecuencias  
        for (int i = 0; i < texto.length(); i++) {  
            char c = texto.charAt(i);  
            frecuencias.put(c, frecuencias.getOrDefault(c, 0) + 1);  
        }  
  
        // Segundo recorrido: buscar el primer carácter no repetido  
        for (int i = 0; i < texto.length(); i++) {  
            char c = texto.charAt(i);  
  
            if (frecuencias.get(c) == 1) {  
                return c;  
            }  
        }  
  
        return null;  
    }  
}
```

```
public static void main(String[] args) {

    System.out.println(primerCaracterNoRepetido(texto: "barco"));
    System.out.println(primerCaracterNoRepetido(texto: "abarca"));
    System.out.println(primerCaracterNoRepetido(texto: "aabbcr"));

}
```

```
b
b
c
```

2. **Análisis de Colisiones:** Si se utiliza un HashMap internamente, explique qué ocurre si dos caracteres distintos generan el mismo hashCode() (Colisión de Hash) y cómo la estructura resuelve este problema sin perder información.

Un HashMap utiliza el hashCode() de sus claves para determinar en qué posición interna almacenar cada elemento.

Si dos caracteres diferentes producen el mismo valor de hashCode(), se produce una colisión de hash. Esto no significa que se pierda información, ya que HashMap puede almacenar ambas claves aunque tengan el mismo hash.

Internamente, HashMap utiliza una estructura de buckets. Cuando dos claves terminan en el mismo bucket, se almacenan allí diferentes entradas y para distinguirlas, Java utiliza la comparación de las claves mediante equals().

Consigna 4: Caso Teórico de Borde y Colisiones

Considere las siguientes dos instancias de String:

- String s1 = "FB";
- String s2 = "Ea";

Sabiendo que s1.hashCode() == s2.hashCode() devuelve true (ambos dan el entero 2236)

Para verificar la colisión y la diferencia lógica, ejecutamos el siguiente código de prueba:

```
public class PruebaHash { no usages 1 inheritor
    public static void main(String[] args) { no usages
        String s1 = "FB";
        String s2 = "Ea";

        System.out.println("Hash de 'FB': " + s1.hashCode());
        System.out.println("Hash de 'Ea': " + s2.hashCode());
        System.out.println("¿Son iguales los hashes? " + (s1.hashCode() == s2.hashCode()));
        System.out.println("¿'FB' es igual a 'Ea' con equals? " + s1.equals(s2));
    }
}
```

```
Hash de 'FB': 2236
Hash de 'Ea': 2236
¿Son iguales los hashes? true
¿'FB' es igual a 'Ea' con equals? false
```

1. Si guardamos ambas cadenas en un `HashSet<String>`, ¿cuántos elementos contendrá la colección? Explique cómo utiliza Java los métodos `hashCode()` y `equals()` internamente para determinar si `s2` debe ingresarse o desecharse.

El `HashSet` contendrá 2 elementos ("FB" y "Ea").

El proceso interno que sigue Java al intentar agregar Ea es el siguiente:

1. Calcula "Ea.hashCode()", que da 2236
2. Con ese valor, determina el bucket (cajón) donde debe buscar. Como "FB" ya estaba en ese bucket (tiene el mismo hash, 2236), Java no asume que son iguales
3. Toma el objeto que ya está en el bucket ("FB") y ejecuta "FB.equals("Ea") para compararlos lógicamente, y devuelve false
4. Como equals dio false, Java concluye que "Ea" no es un duplicado. Por lo tanto, lo guarda también en el mismo bucket, pero enlazado al lado de "FB" (formando una lista enlazada o árbol, según la versión de Java).

Por eso, al final el Hashset tiene tamaño 2, como se confirma en la siguiente captura de pantalla (es la continuación del código que probamos arriba) :

```
Set<String> conjunto = new HashSet<>();
conjunto.add(s1);
conjunto.add(s2);
System.out.println("\n PRUEBA EN HASHSET");
System.out.println("Tamaño del HashSet: " + conjunto.size());
System.out.println("Elementos adentro: " + conjunto);
}
```

```
PRUEBA EN HASHSET
Tamaño del HashSet: 2
Elementos adentro: [FB, Ea]
```

Bucket del HashSet (hash=2236)

["FB"] ---> ["Ea"]

(Están enlazados porque son distintos según equals)

2. ¿Cómo afecta la acumulación de colisiones de hash al rendimiento teórico de un `HashMap` en el peor escenario posible (de $O(1)$ a $O(n)$ o $O(\log n)$)?

Cuando muchos objetos tienen el mismo hashCode() (colisiones masivas), todos caen en el mismo bucket, arruinando la eficiencia:

- Sin colisiones (mejor caso):

Java va directo al bucket y encuentra el objeto rápidamente. Complejidad $O(1)$ (tiempo constante).

- Con muchas colisiones (peor caso):

- En Java 7 y anteriores:

Los objetos se apilan en una lista enlazada dentro del bucket.

Si hay 1000 objetos, Java puede tener que revisar los 1000 usando equals(). La búsqueda se vuelve $O(n)$ (lineal, lentísimo).

- En Java 8 en adelante:

Cuando un bucket supera los 8 elementos, Java convierte la lista enlazada en un Árbol Rojo-Negro (árbol binario balanceado). Así, aunque haya 1000 objetos, la búsqueda baja a $O(\log n)$. Sigue siendo más lento que $O(1)$, pero más rápido que $O(n)$.